POSTGRESQL & SQL

Database Fundamentals

FROM FIRST PRINCIPLES TO BUSINESS SYSTEMS

WEEK 01 / LECTURE 01

Introduction to Databases and Database Management Systems

90-minute university lecture | Beginner level

Anar Ahmadov

Data Scientist | AI Engineer

LINKEDIN PROFILE ↗

LECTURE FOCUS

Why databases exist • How DBMSs work • SQL vs NoSQL • OLTP vs OLAP

Banking • Insurance • Telecommunications • E-commerce

A. Lecture Overview

Lecture 01 — Introduction to Databases and Database Management Systems. Duration: 90 minutes. Prerequisites: basic computer literacy; no SQL experience required.

Learning objectives

- Define data, information, databases and DBMSs using a practical example.
- Explain at least three reasons businesses prefer managed databases to flat files.
- Trace a request from an application through a DBMS to stored data.
- Distinguish relational and non-relational approaches without assuming one is universally superior.
- Identify suitable OLTP and OLAP workloads in common industries.

Suggested 90-minute lesson plan

Minutes	Activity
0–10	Business problem and student discussion
10–25	Data, databases, DBMS and storage
25–40	Spreadsheets, files and database trade-offs
40–55	Architecture and application interaction
55–70	Relational / non-relational, SQL / NoSQL
70–80	OLTP / OLAP and PostgreSQL
80–90	Guided activities and knowledge check

Expected competency: Students can explain database choices and system architecture in plain English before learning SQL syntax.

B. Conceptual Introduction — Why Databases Exist

A payment company with 100,000 customers

Imagine a payment service that receives thousands of payments every hour. Each payment has an amount, a customer, a time, a service provider and a status. A spreadsheet may be useful for a small report, but multiple cashiers and applications cannot safely treat one shared spreadsheet as a transactional system of record.

The business must prevent duplicate payment records, preserve correct balances, control access, recover from failures and answer questions such as “How much was paid today?” A database management system provides tools to manage these needs reliably.

Everyday analogy

A database is like a well-organized library collection; the DBMS is the librarian and management system that catalogs items, enforces borrowing rules, controls who can access them and helps find them quickly. The analogy is useful, but real DBMSs also coordinate simultaneous operations and durable storage.

C. Detailed Theory

1. Data, information and databases

Data are recorded facts such as a customer ID, 25.00 AZN, a timestamp or “completed”. Information is data interpreted in context: “Customer 104 completed three payments totaling 75 AZN today.” A database is an organized collection of related data maintained for retrieval and modification.

Purpose: store related facts consistently and make them useful over time. How: define structures, record values, establish relationships and retrieve relevant subsets. Misconception: a database is not necessarily a single table, nor does every database use SQL.

Raw field	Example	Business meaning
customer_id	104	Identifies the payer
amount	25.00	Payment value
status	completed	Payment outcome
paid_at	2026-11-04 14:30	Event time

2. What is a DBMS?

A Database Management System (DBMS) is software that lets applications and authorized users define, store, query, update and protect data. PostgreSQL is a relational DBMS. The DBMS handles concerns such as concurrency, transactions, access permissions, backups and query execution.

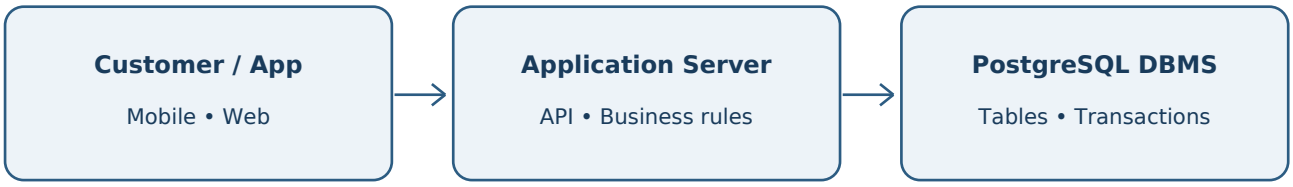
- Definition: the database is the managed data; the DBMS is the software managing it.
- Purpose: reduce inconsistencies, coordinate users and protect data integrity.
- Mechanism: interpret queries, plan operations, read/write storage and return results.
- Misconception: the DBMS does not automatically guarantee a good schema or correct business logic.

3. Why not just use Excel or CSV?

Criterion	Spreadsheet / flat file	DBMS
Small ad hoc analysis	Convenient	Possible, with more setup
Many simultaneous writers	Difficult to coordinate safely	Transactions and concurrency control
Relationships	Manual links / lookups	Keys and relational constraints
Permissions	Often file-level / limited	Role- and object-level controls
Recovery	Depends on file workflow	Backup and recovery mechanisms
Complex repeatable queries	Possible but cumbersome	Declarative query language

Important nuance: spreadsheets are excellent for exploratory analysis and small operational workflows. CSV is a useful interchange format. Databases are preferable when reliable shared state, integrity and controlled concurrent access matter.

4. How an application communicates with a database



Request → validation → query → result → response

The app talks to the DBMS, not directly to disk files.

A customer submits a payment in a mobile app. The application server validates the request and uses a database connection to send operations to PostgreSQL. PostgreSQL checks constraints and permissions, performs the transaction, then returns a result. The application sends a user-friendly response. Real architectures may also include caches, queues and replicas.

A payment request, step by step

- 1. Client sends payment details to the application API.
- 2. Application authenticates the user and checks business rules.
- 3. Application starts database operations; the DBMS checks constraints.
- 4. On success, the transaction commits; on failure, changes can be rolled back.
- 5. API returns a confirmation or error to the client.

5. Relational and non-relational databases

A relational database represents data in relations (commonly displayed as tables) and supports well-defined relationships and constraints. A non-relational database may organize data as documents, key-value pairs, graphs or wide-column structures. Both families can be useful; the choice depends on workload and consistency needs.

Model	Typical representation	Example workload
Relational	Customers, orders, payments tables	Core banking ledger
Document	JSON-like product documents	Variable product catalog
Key-value	Key → value	Session or cache lookup
Graph	Nodes and edges	Fraud relationship analysis

Common misconception: “NoSQL means no SQL” is misleading. The term often means “not only SQL”; capabilities vary by product. PostgreSQL also supports JSONB, so relational and document-style storage are not mutually exclusive.

6. SQL versus NoSQL

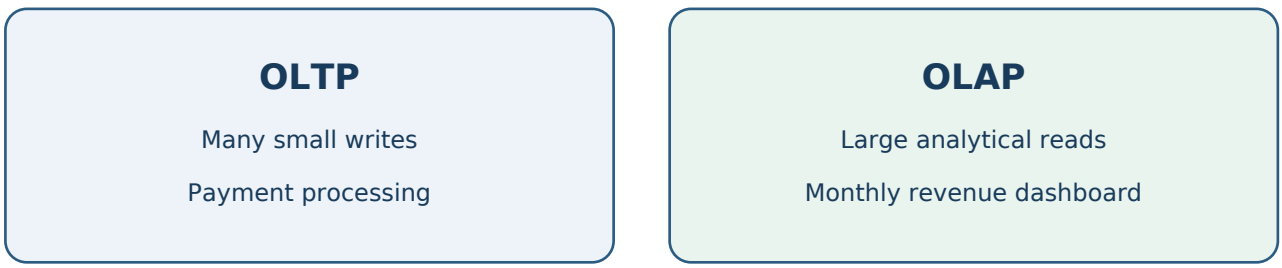
SQL (Structured Query Language) is a language for working with relational data: defining structures, querying records and modifying data. NoSQL describes a broad set of database approaches rather than one alternative language. SQL is not the same thing as PostgreSQL: PostgreSQL is a DBMS that implements SQL and adds PostgreSQL-specific features.

Preview only — a query you will learn in Lecture 3

```
SELECT customer_id, amount
FROM payments
WHERE status = 'completed';
```

This illustrative query asks for customer IDs and amounts of completed payments. The table is conceptual; no SQL execution is required in Lecture 1.

7. OLTP versus OLAP



OLTP (Online Transaction Processing) supports operational actions such as recording payments, changing account balances and creating orders. OLAP (Online Analytical Processing) supports broad aggregations and historical analysis such as monthly revenue trends and churn cohorts.

Dimension	OLTP	OLAP
Primary goal	Fast, correct operations	Decision support
Typical workload	Many short reads/writes	Complex reads over many rows
Example	Process a card payment	Compare regional revenue
Data focus	Current operational records	Historical / integrated data

OLTP and OLAP describe workload patterns, not rigid product categories. PostgreSQL can support both; larger organizations often separate operational databases from analytical warehouses to isolate workloads.

8. PostgreSQL in context

PostgreSQL is an open-source object-relational DBMS known for SQL support, transactions, constraints, extensibility and a rich data-type system. Applications can connect through standard database drivers. PostgreSQL is appropriate for many operational and analytical workloads, but performance depends on schema design, queries, hardware and workload.

- Banking: accounts, transfers and transaction history.
- Insurance: policyholders, policies, claims and payouts.
- Telecommunications: subscribers, tariffs, usage and billing.
- E-commerce: customers, orders, inventory and fulfillment.

D. Visual Explanation — From Events to Insight

An event such as a purchase becomes a stored record. Operational applications read and update these records; analytical queries aggregate many records to produce a KPI. The two activities may use the same DBMS or separate systems.

Event	Stored record	Analysis
Customer pays 20 AZN	Payment ID, customer ID, amount, timestamp	Daily payment volume
Customer cancels a plan	Subscriber ID, cancellation date	Monthly churn rate
Claim is submitted	Claim ID, policy ID, status	Claim approval rate

E. PostgreSQL Code Preview and Expected Result

Lecture 1 does not require SQL proficiency. The following self-contained demonstration is for the instructor to show how a database answers a business question; table creation and queries will be taught later.

```
CREATE TEMP TABLE demo_payments (
  payment_id integer, customer_id integer,
  amount numeric(10,2), status text
```

```
);
INSERT INTO demo_payments VALUES
  (1, 101, 20.00, 'completed'),
  (2, 102, 15.00, 'failed'),
  (3, 101, 30.00, 'completed');
SELECT customer_id, amount
FROM demo_payments
WHERE status = 'completed'
ORDER BY payment_id;
```

customer_id	amount
101	20.00
101	30.00

Explanation: the query excludes the failed payment and returns two completed payments. Common mistake: treating the two rows for customer 101 as accidental duplicates; they represent two different payments.

F. Real Business Use Cases

Fintech — payment integrity

A payment platform must record payment status, amount, customer and provider reliably. A DBMS helps coordinate simultaneous requests and supports auditability. A reporting team can later aggregate completed transactions by date and service.

Insurance — claims management

Claims are connected to customers, policies and inspections. Relational keys reduce ambiguity when one customer holds several policies. Analysts can measure claim frequency, settlement time and repair costs.

Telecommunications — subscriber churn

Billing and service systems track subscribers and usage events. Analytics can identify customers who have not renewed or whose usage has declined. The definition of churn must be explicitly agreed upon before calculation.

E-commerce — orders and inventory

An order involves customers, products, stock and payments. A transactional database helps ensure consistent updates; analytical reporting can reveal best-selling categories and repeat-purchase rates.

G. Guided Practical Exercises

Guided 1 — Choose a storage approach (easy)

Scenario: A small shop records ten product prices and updates them once per week. Steps: (1) Identify users and update frequency. (2) Ask whether simultaneous writes matter. (3) Compare spreadsheet and DBMS setup costs. Suggested reasoning: a spreadsheet may be sufficient today; a DBMS becomes valuable if orders, customers and inventory must be linked and updated concurrently.

Guided 2 — Follow a transaction (medium)

Scenario: A customer pays a utility bill using a mobile app. Steps: identify client, application server and DBMS; explain validation; identify what must be stored; discuss what happens when a payment fails. Expected explanation: the application coordinates business logic, while the DBMS manages durable records and transaction boundaries.

Guided 3 — Classify workloads (medium)

Scenario: A telecom company records a top-up and also prepares a twelve-month revenue dashboard. Steps: identify the real-time write; identify the historical aggregate; label OLTP and OLAP; discuss why large reports may be isolated from live transaction processing. Result: top-up = OLTP; dashboard = OLAP.

H. Independent Student Tasks — No Solutions

Easy — 3 tasks

- E1. Define data, information, database and DBMS in one sentence each. Provide one payment-related example.
- E2. List three limitations of using one shared spreadsheet for a busy online store.
- E3. Name the client, application server and database in an online food-ordering scenario.

Medium — 3 tasks

- M1. Compare a relational database and a document database for a product catalog with changing attributes. State two trade-offs.
- M2. Classify six activities as OLTP or OLAP: account transfer, monthly revenue report, create order, annual churn study, inventory update, regional sales comparison.
- M3. Draw a client–server–DBMS diagram for an insurance claim submission and label the request/response flow.

Challenging — 2 tasks

- C1. Design a conceptual data inventory for a fintech app: customers, payments and service providers. Identify potential relationships and two integrity risks; no SQL needed.
- C2. A company wants a single database for live payments and intensive historical dashboards. Propose a high-level architecture and explain one operational risk and one mitigation.

I. Knowledge Check — Five Short Questions

- 1. What is the difference between a database and a DBMS?
- 2. Why is concurrency important in a payment system?
- 3. Is PostgreSQL a programming language or a DBMS?
- 4. Which workload is usually OLTP: recording a payment or annual revenue analysis?
- 5. Does NoSQL always mean that a database cannot support SQL-like queries?

J. Lecture Summary

Key takeaways

- A database organizes related facts; a DBMS manages access, changes and reliability.
- Databases become especially valuable when data is shared, linked and updated concurrently.
- Relational and non-relational models solve different design problems.
- SQL is a language; PostgreSQL is a DBMS that supports it.
- OLTP focuses on operational transactions; OLAP focuses on analytical questions.

Vocabulary and SQL preview

Core terms: data, information, database, DBMS, relational model, SQL, NoSQL, transaction, concurrency, OLTP, OLAP. SQL keywords seen: CREATE, INSERT, SELECT, FROM, WHERE, ORDER BY. These are only introduced here, not assessed as syntax.

Common mistakes

Confusing SQL with PostgreSQL; assuming spreadsheets are always inappropriate; treating NoSQL as a single technology; assuming OLTP and OLAP require different database brands; believing a DBMS automatically fixes poor application design.

Next lecture: Relational Database Architecture and PostgreSQL Fundamentals — tables, primary and foreign keys, ER diagrams, data types, normalization and your first PostgreSQL database.